

第 6 章

Cloudflare Workers と 生成 AI

Cloudflareでの生成AI活用

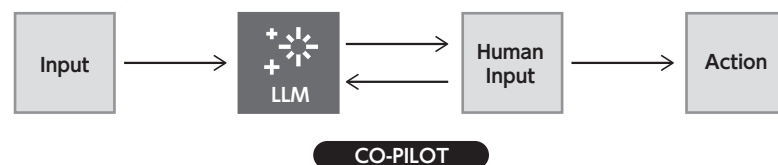
2025年は、Cursor、Claude Code、Codex CLIなどのコーディングエージェントをはじめ、AIエージェントの実用化が進み、生成AIを活用した製品の将来性が強く認識されるようになりました。今では、新規にAIエージェントを活用したサービスを発表する企業が次々現れていますし、まだそうした製品を出していない企業でも、自社のサービスを生成AIによってより便利に使えるようにするための研究などを活発に行っている状況ではないかと思います。

Cloudflareは、この時流にいち早く対応し、生成AIを活用したアプリケーション開発に必要な製品を数多く提供しています。それらは、統合された1つの製品としてではなく、組み合わせたり、部分的に使うことでも価値を発揮できるように設計されています。

AIエージェントとは

生成AIの活用という観点で、真っ先に思い浮かぶのは、AIエージェントでしょう。AIエージェントとよく対比されるのはAI Co-pilotです（図1）。AI Co-pilotは、ユーザーからのプロンプト入力に対して、次取るべき行動を検討しますが、その行動を自律的には行わず、常にユーザーに続行の可否について質問します。そして、承認された場合のみにおいて行動します。このような状態は、完全に自律的に動作しているとは言えません。

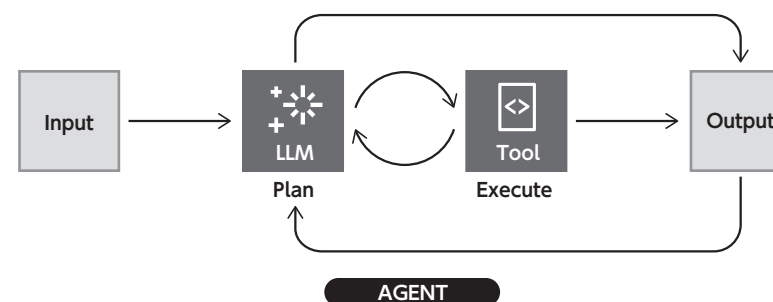
図1 AI Co-pilotの動作フロー^{注1}



注1 <https://developers.cloudflare.com/agents/concepts/what-are-agents/> より引用。

一方で、AIエージェントは、自律的に判断を行い、ユーザーによる確認なしに行動計画を立て、実施します。AIエージェントはツールを持ち、それを通じてAIエージェントの外部に作用します。ツールによって行われる操作には、たとえば、ファイル操作、外部コマンドの実行、スクリプトの実行、インターネットを通じた外部サービスのAPI呼び出しなどが含まれます。AIエージェントに組み込まれたLLMに、ユーザーへの応答や、自身の行動を決定するのに必要な知識がなければ、ツールを通じてローカルのDBや、インターネットなどの外部の情報源から収集します（図2）。AIエージェントの実装によっては、危険と考えられる操作についてはユーザーの確認を求めるものもありますが、これは必須ではありません。たとえば、コーディングエージェントのClaude Code^{注2}は、ユーザーへのすべての確認をスキップする、`--dangerously-skip-permissions`というフラグ^{注3}を持っています。

図2 AIエージェントの動作フロー^{注4}



Cloudflareには、ここで紹介したような、自律的に判断し行動できる、AIエージェントを実装するための製品群がそろっています。

注2 <https://www.claude.com/ja-jp/product/claude-code>

注3 <https://code.claude.com/docs/en/cli-reference#cli-commands>

注4 <https://developers.cloudflare.com/agents/concepts/what-are-agents/> より引用



Cloudflareの生成AI関連製品

Cloudflareの公式ドキュメントのDocs directory^{注5}では、2025年11月現在、以下の製品がAI関連製品として列挙されています^{注6}。

- Agents SDK
- Workers AI
- Vectorize
- Browser Rendering
- Sandbox SDK
- AI Gateway
- AI Search
- AI Crawl Control

もちろんですが、CloudflareでのAIエージェント開発はCloudflare Workersを基盤とするので、ここに挙げられていない、本書で紹介した製品も組み合わせる使うことができます。

列挙されている製品のうち、AIエージェントの開発で特に活用できるものを簡単に紹介します。

Agents SDK^{注7}は、Cloudflare WorkersでのAIエージェント開発のコアとなるSDKです。Agents SDKは、TypeScriptによるAIエージェント開発で最もよく使われている、VercelのAI SDK^{注8}と組み合わせて使えます。そのうえで、セッション管理などの難しい点を、Durable ObjectsなどのCloudflare製品を組み合わせるうまく隠蔽することで、簡単に実現してくれます。Durable Objectsベースのしくみなので、ユーザーごとの個別の会話履歴などのデータを保持できます。Agents SDK自体のAPIは比較的薄いので、学習コストが低

くなります。特に、過去にVercel AI SDKを使ってAIエージェント開発をしたことのある方であれば、Cloudflareに移行するのは簡単ではないかと思います。また、Agents SDKでは、MCPサーバーの実装に利用できるMcpAgent^{注9}クラスも配布しています。

Workers AI^{注10}は、Cloudflareのグローバルネットワーク上のサーバーレスGPUでホストされるAIモデルを実行できる製品です。ホストされるのはオープンソースのモデルで、OpenAIのgpt-ossや、MetaのLlama 4など、著名なモデルがカバーされています。ホストされている中には、AIエージェントが機能呼び出し（Function calling）を行うのに使えるモデルもあります。また、文書をベクトルデータベースに格納するのに利用できる埋め込みモデルもホストされています。たとえば、日本語向けの埋め込みモデルとして著名な、Preferred Networksのplamo-embedding-1b^{注11}がホストされています。Workers AIがホストするモデルは、Cloudflare Workersからであればバインディングオブジェクトのメソッド経由で、外部からであればCloudflareのREST API経由で呼び出せます。オープンソースでない、AnthropicのClaude Sonnetなどのモデルを使用したい場合は、Workers AIではなく、Vercel AI SDKのProvider^{注12}などを使用するのがよいでしょう。

Vectorize^{注13}は、Cloudflareのホストするベクトルデータベースです。ベクトルデータベース^{注14}は、データを「意味」を表すベクトルに変換して保存し、クエリの意味に近いデータを高速に検索できるデータベースエンジンです。クエリのベクトルと保存されたベクトルの類似度を計算し、スコアが高い順に返すしくみになっています。単にドキュメントの検索データベースとして使用できますし、AIエージェントと組み合わせて、LLMが持っていない知識を検索して補う、検索拡張生成（RAG）に活用することもできます。AIエージェントの文脈では、会話履歴の圧縮目的で、過去のメッセージをベクトル化して保存し、必要

注5 <https://developers.cloudflare.com/directory/>

注6 <https://developers.cloudflare.com/directory/?product-group=AI>

注7 <https://agents.cloudflare.com/>

注8 <https://ai-sdk.dev/>

注9 <https://developers.cloudflare.com/agents/model-context-protocol/mcp-agent-api/>

注10 <https://developers.cloudflare.com/workers-ai/>

注11 <https://developers.cloudflare.com/workers-ai/models/plamo-embedding-1b/>

注12 <https://ai-sdk.dev/providers/ai-sdk-providers>

注13 <https://developers.cloudflare.com/vectorize/>

注14 <https://www.cloudflare.com/ja-jp/learning/ai/what-is-vector-database/>

になったタイミングで検索して取り出す使い方もあるでしょう。長期にわたって記憶を維持するAIエージェントを開発するには、コンテキストウィンドウの節約のために、会話履歴の圧縮が必要になりますので、非常に重宝する製品です。前述のとおり、格納するベクトルの生成には、Workers AIがホストする埋め込みモデルが使えます。もちろん、Workers AI以外でホストされるモデルで生成したベクトルを格納しても構いません。

Browser Rendering^{注15}は、CloudflareのホストするヘッドレスのChromeを操作する製品です。レンダリングされたページはHTMLそのままとしても、Markdownとしても取得でき、AIエージェントにWebサイトの解析をさせたい場合に最適です。そのほかにも、PDFの作成など、ヘッドレスのChromeでできる操作の多くがカバーされています。PuppeteerやPlaywright⁰のBrowser Rendering対応版が配布されています^{注16}、同様にBrowser Rendering向けのPlaywright MCP^{注17}も配布されています。このMCPサーバーは簡単に自分のCloudflare環境にデプロイできるので、AIエージェントの実装目的にも、ClaudeのWeb版などのMCPホストからヘッドレスのChromeを動かす目的にも利用できます。

Sandbox SDK^{注18}は、2025年11月現在ベータ版として公開されている、Cloudflare上でホストされた、セキュアかつ、隔離されたコード実行環境を、Cloudflare Workersから操作するためのSDKです。基盤としてCloudflare Containers^{注19}を利用しており、通常のLinux環境 (Ubuntu) 上で動作します。コマンドとしては、事前にインストールされた、curl、git、unzip、zip、jqなどが使えます。また、ファイルシステムの操作に対応しているので、Workerからのディレクトリやファイルの作成や読み取りなどが容易に行えます。コードを動かしたい場合は、JavaScript、TypeScript、Pythonを簡単に実行できます。Sandboxに使用するコンテナのDockerfileは自由に変更可能なので、他のプロ

注15 <https://developers.cloudflare.com/browser-rendering/>

注16 <https://developers.cloudflare.com/browser-rendering/puppeteer/> <https://developers.cloudflare.com/browser-rendering/playwright/>

注17 <https://developers.cloudflare.com/browser-rendering/playwright/playwright-mcp/>

注18 <https://developers.cloudflare.com/sandbox/>

注19 <https://developers.cloudflare.com/containers/>

グラミング言語が必要であれば、そのランタイムを追加してもよいでしょう。Sandbox SDKは、AIエージェントの書いた、信頼できないコードを実行するための環境として活用できます。

紹介した製品について簡単にまとめると、Agents SDKがCloudflare WorkersによるAIエージェント実装のコアとなり、そこにWorkers AI（または外部のモデル）を組み込んで、思考や、機能呼び出しを実現できます。長期記憶はVectorizeによって保持し、外部から情報を収集する必要がある場合は、Browser Renderingも使えます。コードを実行して正確な処理を行う必要があれば、Sandbox SDKを使えます。このように、それぞれ独立した製品を協調させて使うことにより、Cloudflareの基盤を活かした多機能なAIエージェントを開発できます。

Agents SDKの詳細と利用方法

先ほど紹介したAgents SDKについてももう少し踏み込んで、具体的な実装での利用方法について解説します。

Agents SDKは、npmにてagentsという非常にシンプルな名前のパッケージとして配布されています。このような一般的な名前なので、Cloudflare以外でも使える汎用のSDKかのように思えますが、Cloudflare専用です。

agentsパッケージをインストールすると、そこからAgentクラスをインポートできます。Agents SDKによるエージェントの実装は、このAgentクラスを継承したクラス（以下、Agentのサブクラスと呼びます）を宣言することで行います。なお、Agents SDKはAIエージェントの実装に限らず利用可能なため、ここでは、単にエージェントと呼びます。このAgentクラスは、Durable ObjectsのDurableObjectクラス^{注20}を継承したものであり、通常のDurable Objectsと同様の形で動作します。そのため、厳密に理解したい場合は、Durable Objectsについて事前に調べてからのほうが取っ付きやすいと思います。更に厳密に言うと、Agentクラスは、DurableObjectクラスを継承した、PartyKit^{注21}

注20 <https://developers.cloudflare.com/durable-objects/api/base/>

注21 <https://github.com/cloudflare/partykit>

のServerクラスを継承しています。

エージェントは、それぞれがDurable Objectsのインスタンスであるため、独立したストレージを持ちます。たとえば、AIエージェントを実装する場合、インスタンスのストレージにチャットの履歴を保存する形になるでしょう。インスタンスは、名前（識別子）によって識別されるので、こういったルールで名前を付けるかが重要となります。クライアント側から自由に接続先エージェントのインスタンス名を指定できるようにつくりにすると、本来閲覧できてはいけない情報をエージェントが返す可能性があります。必要に応じて、認証を挟むなどの実装をするよう注意しましょう。

Agents SDKにおけるエージェントとは何なのか？

Agents SDKのAgentクラスの具体的な使い方に触れる前に、CloudflareのAgents SDKで構築できるエージェントについて簡単に概要を説明します。

先ほど述べたとおり、Agents SDKはAIエージェントに限らず利用可能です。そのため、エージェントの大枠をつかむためには、AIエージェントをはじめとするエージェント構築に必要とされる機能は何なのか、という観点から理解する必要があります。筆者の認識における、Agents SDKのエージェントの主要な機能は、以下の3つです。

1. セッション維持
2. エージェント・クライアント間RPC
3. stateのエージェント・クライアント間での同期

1つめの「セッション維持」についてはわかりやすいのではないかと思います。たとえば、AIチャットのエージェントであれば、確実にチャットの履歴が必要となります。ここでは、このチャット履歴を維持する機構のことをセッションと呼びます。エージェントのインスタンスひとつひとつが、セッションを保持していると理解するとよいでしょう。チャットのセッションが維持される期間は、短期のこともあるでしょうし、ユーザーが途中まで送信したチャットを1週間以上後に思い出して再開するような、長期にわたることもあるでしょう。また、セッションが分けられる単位もさまざまです。まず、最低限の単位とし

てユーザー単位があるでしょう。複数のユーザーのチャット履歴が混ざったら問題です。そのうえ、よくあるAIチャットのアプリケーションでは、1人のユーザーが、会話履歴の混ざらないチャットのセッションを好きなだけ作成できるようになっています。この場合は、セッションの分離単位がユーザー単位だけでは不足します。このように、エージェントのセッションは、さまざまな期間、および分離単位が考えられます。ここではチャットを例に取り上げましたが、それぞれのセッションが、独自の形式のデータを維持できると考えたら、その応用幅について想像しやすいのではないのでしょうか？

2つめの「エージェント・クライアント間RPC」は、エージェントを活用したアプリケーションの実装を簡単にしてくれます。たとえば、チャットでAIエージェントとやりとりするWebアプリケーションの例を考えてみます。Webアプリケーション側から、AIエージェントにチャットのメッセージを送信するには、本来そのためのAPIを用意し、サーバー・クライアントのそれぞれで、それなりの量の実装が必要となります。ここで、Agents SDKを利用すると、Agentのサブクラスに定義されたメソッドを、クライアント側からTypeScriptの型付きで簡単に呼び出せます。サーバ側にある処理を、クライアント側にあるオブジェクトのメソッドかのように呼び出せるのは非常に快適です。

3つめの「stateのエージェント・クライアント間での同期」も重要です。Agents SDKで作成したエージェントのインスタンスは、それぞれが独自のデータを持ちます。1つめの例で挙げた、チャットの履歴もまさにその1つです。チャットの例で言うと、基本的には、チャットのメッセージを送信すると、それに対する応答のメッセージが返る、一対一の対応関係が基本となります。一方、チャットのメッセージを処理するエージェントは、単純な応答だけではなく、時間のかかる処理をする可能性もあります。たとえば、エージェントにインターネットでの調査を依頼するようなケースでは、エージェントが今、どんな情報を検索しているかといった進捗状況の共有がないと、ユーザーが不安になってしまうかもしれません。そのような場合に、stateの同期機能を活用すると、エージェント・クライアント間がWebSocketで接続され、エージェント側でstateが更新されるたびに、それをクライアント側のUIに反映することができます。

これらの3つの主要な機能は、まさにAIチャットのエージェントを実装する

には最適でしょう。しかしながら、ひとつひとつの機能は、AIの利用に関係なく活用可能なものです。ぜひ、ご自身のアプリケーション実装のアイデアの源として、引き出しに入れておいてください。

Agentクラスの使い方

Agents SDKのAgentクラスにより構築したエージェントには、主に3つの呼び出し方法があります。

1. HTTPリクエスト
2. WebSocket
3. Workerからの呼び出し

HTTPリクエストおよびWebSocketを使った呼び出しは、Agents SDKのClient APIを使うもので、主にブラウザから行います。Agentのサブクラスには、HTTPリクエストやWebSocket接続を受け付けたときに動作する、事前に決められた名前のメソッドを宣言できます。また、それとは別に、自由に宣言したメソッドをブラウザから呼び出すための特別なRPC機構もあります（これについては後ほど紹介します）。

Workerからの呼び出しは、基本的に通常のDurable Objectsの利用方法と変わりません。Durable Objectsインスタンスのメソッドは、もともとWorker内からRPC呼び出し可能^{注22}であるため、Agents SDKが用意した特別なRPC機構は利用しません。

HTTPリクエスト/WebSocketでのエージェント利用

HTTPリクエストおよびWebSocketで利用する場合は、HTTPリクエストの場合にonRequest、WebSocketの場合にonConnectやonMessageというハンドラーのメソッドを、Agentのサブクラスに実装します。これらのメソッドを実装したクラスをexport defaultで公開するようにドキュメントに記載され

^{注22} <https://developers.cloudflare.com/durable-objects/best-practices/create-durable-object-stubs-and-send-requests/>

ています^{注23}。しかしながら、筆者の環境では、export defaultによるAgentサブクラスの公開での動作が確認できておらず、また、Agents SDKのexampleにも直接export defaultで公開する例は含まれていませんでした。代わりに、自分で実装したWorkerのfetchハンドラー内でrouteAgentRequest^{注24}という関数呼んでエージェントにルーティングする例が示されていたので、こちらの方法を記載します（リスト1）。

リスト1 HTTPリクエストおよびWebSocketを使うAIエージェントの例

```
import { Agent } from "agents";

export class MyAgent extends Agent {
  async onRequest(request: Request): Promise<Response> {
    // HTTPリクエストを受け取るために実行
    return new Response( "HTTPリクエストを受信"); // レスポンスをここで返す
  }

  async onConnect(connection: Connection, ctx: ConnectionContext) {
    // WebSocketのコネクションを確立するために実行
  }

  async onMessage(connection: Connection, message: WSMMessage) {
    // WebSocketのメッセージを受け取るために実行
    connection.send("メッセージを受信"); // メッセージをここで送り返す
  }
}

export default {
  async fetch(request: Request, env: Env, _ctx: ExecutionContext) {
    return (
      // routeAgentRequest
      (await routeAgentRequest(request, env)) ||
      new Response("Not found", { status: 404 })
    );
  }
} satisfies ExportedHandler<Env>;
```

^{注23} <https://developers.cloudflare.com/agents/api-reference/agents-api/>

^{注24} <https://developers.cloudflare.com/agents/api-reference/calling-agents/#authenticating-agents>

routeAgentRequestによって公開されたエージェントには、AgentClientクラスや、React向けのuseClient hookなどのClient API^{注25}を使って接続します。これらのClient APIの機能を使うと、エージェントの持つstateを、WebSocket経由でリアルタイムにクライアントに同期できます（stateについては後述します）。接続先のエージェントの種類の識別はエージェントのクラス名を、インスタンスの識別については、インスタンス名をパラメータに指定します（**リスト2**）。

リスト2 AgentClientの使用例

```
import { AgentClient } from "agents/client";

const client = new AgentClient({
  agent: "my-agent", // kebab-caseにしたエージェントのクラス名
  name: "agent-001", // 特定のインスタンス名
  host: window.location.host, // 接続先ホスト(ここでは同じホストを使用)
});

client.onmessage = (event) => {
  // エージェントからのメッセージ受信時に実行
};

// エージェントにメッセージを送信
await client.send("Hello, from client!");
```

AgentClientによるエージェントの呼び出しには、sendメソッド^{注26}を使います。sendメソッドでは単純な文字列やArrayBuffer等を送信でき、送られたデータがAgentのサブクラスのonMessageハンドラーに届きます。エージェントに対して、構造化されたデータを送信したい場合は、JSON.stringifyを使うとよいでしょう。

React向けのuseAgent hookを使用する場合は、sendメソッドによる呼び出しは行いません。代わりに、Agentのサブクラスに@callable()のデコレータ付きでメソッドを宣言し、クライアント側からRPC形式で呼び出します。これに

注25 <https://developers.cloudflare.com/agents/api-reference/agents-api/#client-api>

注26 <https://developers.cloudflare.com/agents/api-reference/agents-api/#connection>

ついては、公式example内のチェスアプリの例^{注27}が参考になるので**リスト3**に引用します。

リスト3 Agentサブクラスの@callable()付きのメソッド宣言例

```
import { Agent, callable } from "agents";

export class ChessGame extends Agent<Env, State> {
  @callable()
  join(params: { /* 中略 */ }) {
    // ゲームに参加する
  }

  @callable()
  leave() {
    // ゲームから離脱する
  }
}
```

わかりやすさのために、詳細な実装については削っている点にご留意ください。**リスト3**では、joinやleaveというメソッドがそれぞれ@callable()デコレータによってマークされています。

Reactアプリケーション側の実装を見ると、Agentのサブクラスの宣言で@callable()によりマークされたjoinメソッドをuseAgent hookの呼び出しで初期化したstub変数に対して呼び出していることがわかります（**リスト4**）。

リスト4 useAgentによるjoinメソッドへのRPC呼び出し例

```
function App() {
  const playerId = usePlayerId();

  const { stub } = useAgent<ServerState>({
    host,
    name: activeGameName,
    agent: "chess",
  });

  useEffect(() => {
    (async () => {
```

注27 <https://github.com/cloudflare/agents/tree/agents@0.2.26/openai-sdk/chess-app>

```

    try {
      // joinメソッドを呼び出せる
      const res = (await stub.join({ playerId, preferred: "any" })) as
JoinReply;
      if (!res?.ok) return;
      // OKの場合の処理
    } catch (error) {
      // エラーの場合の処理
    }
  }();
}, [/* 中略 */]);

return <div>{ /* 中略 */ }</div>;
}

```

このとき、useAgentに渡す引数の内容は、AgentClientクラスを初期化するときの内容と同じです^{注28}。

AgentClientやuseAgentはWebSocketでエージェントに接続しますが、WebSocketを使わず、単にHTTPリクエストを送りたい場合はagentFetch^{注29}関数を使用します。

Workerからのエージェントの利用

Workerのコード内から直接エージェントを呼び出す場合は、Client APIで必要だったAgentサブクラスへのルーティングが不要となるので、routeAgentRequestを使う必要がありません。この場合、エージェントのインスタンスはgetAgentByName関数^{注30}で取得します（リスト5）。

リスト5 Workerからエージェントのメソッドを呼び出す例

```

import { Agent, AgentNamespace, getAgentByName } from 'agents';

interface Env {
  // MyAgentの型情報を利用できるよう、Envに設定

```

注28 実際のチェスアプリの例では、onStateUpdateという、stateが更新されるたびに呼ばれるメソッドもuseAgentの引数のオブジェクトに設定されていますが、ここでは割愛しています。

注29 <https://developers.cloudflare.com/agents/api-reference/agents-api/#agentfetch>

注30 <https://developers.cloudflare.com/agents/api-reference/calling-agents/#calling-methods-on-agents>

```

MyAgent: AgentNamespace<MyAgent>
}

export default {
  async fetch(request, env, ctx) {
    // エージェントのインスタンスを取得
    const myAgent = getAgentByName<Env, MyAgent>(
      env.MyAgent, // エージェントのバインディング
      'my-agent-001', // 特定のインスタンス名
    );
    // エージェントの chat メソッドを呼び出す
    const chatResp = myAgent.chat('Hello!');
    // エージェントから受け取ったメッセージを返す
    return Response.json({
      message: chatResp,
    });
  },
} satisfies ExportedHandler<Env>

export class MyAgent extends Agent {
  // @callable() デコレータは不要
  async chat(prompt: string) {
    // (ここで必要に応じてLLMを呼び出す)
    return "response message";
  }
}

```

Client APIでは、エージェントの識別のために、そのクラス名を指定していましたが、getAgentByNameでは、fetchハンドラーに渡ってくるenv変数から取得できる、Durable Objectsのバインディングをそのまま受け渡しています。getAgentByNameで指定するインスタンスの名前は、AgentClientの初期化やuseAgentの引数で渡していたものと同じです。取得したエージェントのインスタンスには、通常のDurable Objectsと同じようにRPC形式でメソッド呼び出しができます。Client APIが不要な場合は、このような単純な利用方法も選択肢に入ります。

エージェントのstate管理

各エージェントのインスタンスは、それぞれが独自のstate^{注31}を持ちます。

注31 <https://developers.cloudflare.com/agents/api-reference/store-and-sync-state/>

stateは、エージェント自身の実装上で操作できますし、Client APIを通じてブラウザなどから読み書きすることもできます。stateは、エージェントのインスタンスのプロパティとしてアクセスでき、通常のJavaScriptのオブジェクトの形式として扱えます。その更新には、エージェントのsetStateメソッドを使用します。stateは、Durable Objectsのストレージに保存されるため、全世界で一貫性のある値として扱えますし、WebSocketを通じてクライアントに対してリアルタイムに値が同期されます。Durable Objectsのストレージを使用するため、JSON形式にシリアル化/デシリアル化できる値のみが保存できるという制約があります。

具体的な例として、カウンターのエージェントを示します(リスト6)。

リスト6 stateを持つ、カウンターのエージェントの例

```
type State = {
  counter: number;
};

export default class CounterAgent extends Agent<Env, State> {
  initialState = {
    count: 0,
  };
  @callable()
  increment() {
    this.setState({ count: this.state.count + 1 });
  }
  @callable()
  decrement() {
    this.setState({ count: this.state.count - 1 });
  }
}
```

CounterAgentは、初期状態としてcount: 0が指定されています。このエージェントのインスタンスを取得し、incrementおよびdecrementを呼び出すことで、カウンターの値を増減させることができます。CounterAgentはexport defaultで公開されており、それぞれのメソッドに@callableデコレータが設定されているので、Client APIのuseAgentを使って操作することができます。

useAgentの引数にはonStateUpdateのコールバックが設定できるので、

Reactアプリケーションと同期させるには、useStateと組み合わせます。リスト

6では、RPCメソッド経由でstateを更新しましたが、リスト7のようにクライアント側から直接エージェントのsetStateメソッドを呼び出して更新することもできます^{注32}。

リスト7 カウンターのクライアントの例

```
import { useState } from "react";
import { useAgent } from "agents/react";

function App() {
  const [state, setState] = useState({ count: 0 });

  const { stub } = useAgent({
    agent: "counter-agent",
    name: "counter-agent-001",
    onUpdate: (newState) => setState(newState),
  });

  return (
    <div>
      <div>Count: {state.count}</div>
      <button onClick={() => stub.increment()}>+</button>
      <button onClick={() => stub.decrement()}>-</button>
    </div>
  );
}
```

ここでは、stateを使ったエージェントでの状態管理について簡単に紹介しましたが、Durable Objectsのストレージを直接読み書きする、より生に近いSQL API^{注33}も存在します。stateは、クライアントからの操作の自由度が非常に高いので、もう少し読み書きに制限を加えたい場合は、SQL APIの方を利用したほうがよいでしょう。

注32 <https://developers.cloudflare.com/agents/api-reference/store-and-sync-state/#synchronizing-state>

注33 <https://developers.cloudflare.com/agents/api-reference/store-and-sync-state/#sql-api>



おわりに

今回は、Cloudflareの基盤を活用したAIアプリケーションの構築に使える製品について紹介しました。特に、その中心となるAgents SDKについて重点的に解説しました。Agents SDKは、強力な機能を複数備えていますが、その使い方の幅も非常に広く、概念としてつかみにくいところが導入のハードルとなっている印象があります。具体的な生成AI呼び出しについては割愛しましたが、生成AIの呼び出しについては、実はほぼ単純な関数呼び出しとなっており、Vercel AI SDKなど、非常に組み込みが容易となっています。

今回紹介した内容は、Agents SDKを理解するうえでの手がかりとして使える重要な内容を含んでいるので、実際にエージェント実装を試す際は、Cloudflare公式のドキュメントと合わせて参考にしてください。